napari-chat-assistant / System Design

Local imaging workbench · live viewer context · deterministic actions · Ollama assistance

Implementation map | v2.3.4 | committed source at 2ed1876 | 10 November 2026

request / result · optional / fallback · red = boundary

1 User + live viewer
Prompt, code, action, shortcut
Layer names, selection, ROI

2 Dock orchestrator
Intent + follow-up state
Local routing before model

3a Direct / local route
Actions, tools, workflow plans
No model request for matches

3b Ollama fallback
Compact context + prompt profile
Reply / tool / pending code

4 Execute or review
Tool: prepare → execute → apply
Code: validate → review → Run

5 Result in the same workspace
Layers, labels, overlays, tables, widgets
Refresh context + record local diagnostics

Reply → transcript Tool → dispatcher Code → review panel

Viewer state feeds the next turn; the model does not own the viewer.

A / CLIENT & WORKBENCH

napari.yaml → widgets/chat_widget.py

Qt dock widget
Connection + model + Fast mode
Chat transcript + prompt editor

Interaction surfaces
Library / Templates / Actions
Shortcuts / Session / Advanced

Prompt → natural language request
Run My Code → execute user Python after preflight
Refine My Code → model-assisted repair
Actions / Shortcuts → direct, repeatable execution
Optional Voice Input → editable prompt text

B / CONTEXT & SESSION STATE

agent/context.py · profiler.py · session_memory.py

Live viewer context
Names, types, shapes, dtype
Selection, ROI, layer profiles

Conversation state
Intent + follow-up constraints
Recent action + failure context

Compact viewer payload goes to the model.
Layer profiling uses local image information.
Session memory binds follow-ups to prior outcomes.
Pending actions and pending code remain explicit.
Context is refreshed after tools change the viewer.

C / ROUTING & WORKFLOW PLANNING

agent/prompt_routing.py · intent_state.py · workflow_planner.py

Local request handling
UI help + recognized operations
Intent guards + route selection

Workflow routes
Plan / execute / report
Safe viewer-control sequences

Matched local routes can avoid Ollama entirely.
Code / repair intent can skip local workflow routing.
Conservative binary-segmentation plan is supported.
Workflow executor assesses and refines mask quality.
Unmatched requests continue to the model path.

D / LOCAL MODEL SERVICE

agent/client.py · tools.py · widgets/chat_widget.py

Request builder
tool_routing / code_generation
code_repair prompt profiles

Ollama HTTP API
Configured base URL + model
POST /api/chat; stream=false

Payload: compact context + memory + user request.
Background request keeps model I/O off the UI thread.
Fast mode adjusts reasoning for supported models.
Normalized response: reply, tool, or code.
Model loading / unloading and pull progress are available.

E / TOOL EXECUTION CONTRACT

agent/dispatcher.py · tool_registry.py · tool_types.py

1 prepare
Resolve tool name
Bind viewer + args

2 execute
Compute tool result
Immediate or worker

3 apply
Update viewer / UI
Return result message

ToolSpec describes parameters, layer types and mode.
PreparedJob → ToolResult carries computation payloads.
Worker tools compute through napari thread_worker.
Returned results are applied through the UI callback.
Direct actions and model-selected tools share dispatch.

F / IMAGE ANALYSIS & VIEWER TOOLS

agent/tools_builtin/ · image_ops.py · analysis widgets

Pixels + masks
CLAHE / Gaussian / projection
Threshold / cleanup / labels

Measures + layout
ROI values / label tables
Montage / overlays / visibility

NumPy, SciPy, scikit-image and matplotlib support analysis.
Interactive tools: intensity, line profile, group comparison.
Layer outputs stay attached to the live viewer.
Built-ins include inspection and synthetic examples.
Some registered workflow / SAM names are placeholders.

G / PYTHON GENERATION & REVIEW

agent/code_validation.py · pending_code_panel.py

Generate / refine
Normalize code + inspect AST
Layer/API checks + diagnostics

Review → Run Code
Blocked issues prevent running
Warnings / repair feedback

Execution namespace: viewer, selected_layer, np, napari.
exec(compile(...)) runs inside the napari process.
run_in_background(compute, apply) supports heavy work.
Captured stdout and errors appear in the transcript.
Validation is a preflight check, not a security sandbox.

H / WORKSPACE SAVE & RESTORE

agent/workspace_state.py

JSON manifest
Layers + viewer display state
Source references + metadata

Workspace assets
OME-Zarr image / labels data
Inline records where suitable

Capture source information, transforms and layer state.
Generated image data can be stored beside the manifest.
Shapes / points and display properties are serialized.
Restore reopens sources or loads saved layer assets.
This is workspace persistence, not a model memory DB.

I / LIBRARY & LOCAL PREFERENCES

agent/prompt_library.py · ui_state.py · action_library.py

Reusable content
Saved prompts + code snippets
Built-in templates + actions

Local state
prompt_library.json
ui_state.json + shortcuts

Default location: ~/.napari-chat-assistant/
Pin, tag, rename and reuse prompts or code.
Action definitions describe direct execution targets.
UI state remembers layouts and selected preferences.
Templates are curated starting points, not RAG retrieval.

J / OPTIONAL VOICE INPUT

voice/controller.py · audio_recorder.py · backends/

Record → transcribe
Qt multimedia microphone
Local faster-whisper backend

Review → submit
Editable transcript preview
Run via shared prompt callback

Input-device selection + live recording level meter.
Transcription runs in a background worker.
Voice window stays available during viewer work.
Requires optional dependencies and local model assets.
Submitted speech follows the ordinary request routes.

K / OPTIONAL SAM2 SEGMENTATION

agent/sam2_backend.py · integrations/sam2_adapter.py

Prompt geometry
Box / foreground-background points
Interactive preview + 3D tools

Local SAM2 predictor
Configured checkpoint + config
PyTorch; CUDA when available

Adapter prepares 2D numeric grayscale input as RGB.
SAM2 produces segmentation masks for viewer results.
Backend setup checks environment and model assets.
3D propagation is exposed through the tool layer.
SAM2 is a separate optional model from the chat LLM.

L / DIAGNOSTICS & SYSTEM LIMITS

telemetry/ · logging_utils.py · summary.py · report.py

Local observability
assistant.log / crash.log
model_telemetry.jsonl

Reports + controls
Opt-in telemetry UI setting
Latency / tokens / outcomes

Local model by default; base URL is configurable.
Model payload contains text context, not image uploads.
Large arrays and synchronous steps can still block UI.
Undo snapshots cover viewer controls, not all operations.
Uncommitted remote-execution work is outside this map.

Read with system-design.md for request examples, source links, implementation boundaries and regeneration instructions.

Architecture documentation only · no runtime code changes · SVG remains editable and sharp at any zoom